

SWE2020-UC Machine Learning and Big Data — Assignment 3: Tree-Based Modeling

Group Members & Contributions:

1. **Gabriel Odiwuor — 673709**: Led overall notebook structure, preprocessing for both datasets, and model evaluation reporting.
2. **Hope Muiruri — 672983**: Handled exploratory data analysis, visualizations, and interpretation write-ups for the COVID classification task.
3. **Martha Muasya — 673685**: Built and tuned the Random Forest and Gradient Boosting classifiers, including imbalance handling and feature-importance analysis.
4. **Kariuki Rebecca — 670138**: Developed the Decision Tree Regressor for car price prediction, tree interpretation, and the overall discussion/lessons-learned section.

In this assignment we apply tree-based models to two real-world tasks:

- **Task 1 (Classification)**: COVID-19 risk prediction using the provided coronavirus dataset. We train a **Random Forest Classifier** and a **Gradient Boosting Classifier**, compare their performance, and inspect feature importance.
- **Task 2 (Regression)**: Car price prediction using the Auto dataset. We train a **Decision Tree Regressor**, evaluate it with RMSE and R^2 , and interpret the tree structure.

Each part follows the same pipeline: load and explore the data, preprocess it, build baselines, train/tune the required models, evaluate them, and discuss what the results mean.

```
In [1]: # -----  
# Global imports and settings  
# -----  
  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
import warnings  
  
from sklearn.model_selection import train_test_split, GridSearchCV, Stratifi  
from sklearn.preprocessing import LabelEncoder  
from sklearn.compose import ColumnTransformer  
from sklearn.pipeline import Pipeline  
from sklearn.impute import SimpleImputer  
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassif  
from sklearn.tree import DecisionTreeRegressor, plot_tree, export_text  
from sklearn.dummy import DummyClassifier, DummyRegressor  
from sklearn.metrics import (  
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
```

```
confusion_matrix, ConfusionMatrixDisplay, classification_report,  
mean_squared_error, mean_absolute_error, r2_score, root_mean_squared_err  
)  
  
warnings.filterwarnings("ignore")  
sns.set_style("whitegrid")  
plt.rcParams["figure.dpi"] = 100  
RANDOM_STATE = 42  
np.random.seed(RANDOM_STATE)  
pd.set_option("display.max_columns", 40)
```

Part A — Task 1: COVID-19 Risk Classification

Objective: Predict whether an individual is likely to test positive for COVID-19 (covid19_test_results).

Models: Random Forest Classifier and Gradient Boosting Classifier.

A.1 Data Loading and Overview

```
In [2]: df_covid = pd.read_csv("coronavirusdataset.csv")  
print("Shape:", df_covid.shape)  
df_covid.head()
```

Shape: (7294, 45)

Out [2]:

	batch_date	test_name	swab_type	covid19_test_results	age	high_risk_exposure_
0	2020-10-20	SARS-CoV-2, NAA	Nasal	Negative	39	
1	2020-10-20	COVID-19 PCR External Result	Nasal	Negative	56	
2	2020-10-20	Rapid COVID-19 PCR Test	Nasal	Negative	35	
3	2020-10-20	Rapid COVID-19 PCR Test	Nasal	Negative	37	
4	2020-10-20	Rapid COVID-19 PCR Test	Nasal	Negative	42	

5 rows × 45 columns

Interpretation

The dataset has 7,294 rows and 45 columns. Each row is one COVID-19 test record with demographics, risk factors, symptoms, vitals, and lab/imaging fields. The target is `covid19_test_results`.

In [3]: `df_covid.info()`

```

<class 'pandas.DataFrame'>
RangeIndex: 7294 entries, 0 to 7293
Data columns (total 45 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   batch_date                               7294 non-null   str
1   test_name                               7294 non-null   str
2   swab_type                               7294 non-null   str
3   covid19_test_results                    7294 non-null   str
4   age                                      7294 non-null   int64
5   high_risk_exposure_occupation           7294 non-null   bool
6   high_risk_interactions                  2727 non-null   object
7   diabetes                                7294 non-null   bool
8   chd                                      7294 non-null   bool
9   htn                                      7294 non-null   bool
10  cancer                                  7294 non-null   bool
11  asthma                                  7294 non-null   bool
12  copd                                    7294 non-null   bool
13  autoimmune_dis                          7294 non-null   bool
14  smoker                                  7294 non-null   bool
15  temperature                             1869 non-null   float64
16  pulse                                   1866 non-null   float64
17  sys                                      1727 non-null   float64
18  dia                                      1727 non-null   float64
19  rr                                       1544 non-null   float64
20  sats                                    1869 non-null   float64
21  rapid_flu_results                       6 non-null      str
22  rapid_strep_results                     11 non-null      str
23  ctab                                    1288 non-null   object
24  labored_respiration                     1963 non-null   object
25  rhonchi                                 723 non-null     object
26  wheezes                                 961 non-null     object
27  days_since_symptom_onset                 591 non-null     float64
28  cough                                    7294 non-null   bool
29  cough_severity                           178 non-null     str
30  fever                                    3137 non-null   object
31  sob                                       7294 non-null   bool
32  sob_severity                             82 non-null      str
33  diarrhea                                 7294 non-null   bool
34  fatigue                                  7294 non-null   bool
35  headache                                 7294 non-null   bool
36  loss_of_smell                           7294 non-null   bool
37  loss_of_taste                           7294 non-null   bool
38  runny_nose                              7294 non-null   bool
39  muscle_sore                             7294 non-null   bool
40  sore_throat                             7294 non-null   bool
41  cxr_findings                            7 non-null       str
42  cxr_impression                          7 non-null       str
43  cxr_label                              7 non-null       str
44  cxr_link                                7 non-null       str
dtypes: bool(19), float64(7), int64(1), object(6), str(12)
memory usage: 1.6+ MB

```

```

In [4]: print(df_covid["covid19_test_results"].value_counts())
print("\nPositive rate: {:.3f}%".format(
    100 * (df_covid["covid19_test_results"] == "Positive").mean())

```

```

))
sns.countplot(x="covid19_test_results", data=df_covid)
plt.title("COVID-19 Test Results")
plt.show()

```

```

covid19_test_results
Negative    7269
Positive     25
Name: count, dtype: int64

```

Positive rate: 0.343%

Interpretation

The target is **extremely imbalanced**: almost all tests are Negative and only a tiny fraction are Positive. Accuracy alone would be misleading (a model that always predicts Negative would look excellent). We will emphasise **recall, precision, and F1 on the Positive class**, plus ROC-AUC where possible.

```

In [5]: missing_pct = (df_covid.isnull().mean() * 100).sort_values(ascending=False)
missing_pct.head(20)

```

```

Out[5]: rapid_flu_results      99.917741
cxr_link      99.904031
cxr_label     99.904031
cxr_impression 99.904031
cxr_findings  99.904031
rapid_strep_results 99.849191
sob_severity  98.875788
cough_severity 97.559638
days_since_symptom_onset 91.897450
rhonchi       90.087743
wheezes       86.824787
ctab          82.341651
rr            78.831917
sys          76.323005
dia          76.323005
pulse        74.417329
sats         74.376200
temperature  74.376200
labored_respiration 73.087469
high_risk_interactions 62.613107
dtype: float64

```

Interpretation

Several columns are almost entirely missing (chest X-ray fields, rapid flu/strep, severity fields, many vitals). Keeping columns with >70% missing values would force heavy imputation that mostly invents data, so we drop those and keep features that are usable for a school-level tree model.

A.2 Data Cleaning and Preprocessing

```
In [6]: # Drop IDs / free-text / near-empty columns that are not useful for modelling
drop_cols = [
    "batch_date", "test_name", "swab_type",
    "rapid_flu_results", "rapid_strep_results",
    "cough_severity", "sob_severity",
    "cxr_findings", "cxr_impression", "cxr_label", "cxr_link",
    "days_since_symptom_onset", # >90% missing
    "rhonchi", "wheezes", "ctab", "labored_respiration", # mostly missing c
    "rr", "sys", "dia", "pulse", "sats", "temperature", # >70% missing vita
]

df_c = df_covid.drop(columns=drop_cols).copy()

# Target: 1 = Positive, 0 = Negative
df_c["target"] = (df_c["covid19_test_results"] == "Positive").astype(int)
df_c = df_c.drop(columns=["covid19_test_results"])

# Convert object/bool columns cleanly
bool_like = [
    "high_risk_exposure_occupation", "diabetes", "chd", "htn", "cancer",
    "asthma", "copd", "autoimmune_dis", "smoker", "cough", "sob",
    "diarrhea", "fatigue", "headache", "loss_of_smell", "loss_of_taste",
    "runny_nose", "muscle_sore", "sore_throat"
]

for col in bool_like:
    if col in df_c.columns:
        df_c[col] = df_c[col].map({True: 1, False: 0, "True": 1, "False": 0})

# Remaining object columns: high_risk_interactions, fever
for col in ["high_risk_interactions", "fever"]:
    if col in df_c.columns:
        df_c[col] = df_c[col].map({True: 1, False: 0, "True": 1, "False": 0})
        # leave NaN as missing for imputation

print("Remaining columns:", list(df_c.columns))
print("Missing after drop:\n", df_c.isnull().sum())
print("Class counts:\n", df_c["target"].value_counts())
df_c.describe().T
```

Remaining columns: ['age', 'high_risk_exposure_occupation', 'high_risk_interactions', 'diabetes', 'chd', 'htn', 'cancer', 'asthma', 'copd', 'autoimmune_dis', 'smoker', 'cough', 'fever', 'sob', 'diarrhea', 'fatigue', 'headache', 'loss_of_smell', 'loss_of_taste', 'runny_nose', 'muscle_sore', 'sore_throat', 'target']

Missing after drop:

age	0
high_risk_exposure_occupation	0
high_risk_interactions	4567
diabetes	0
chd	0
htn	0
cancer	0
asthma	0
copd	0
autoimmune_dis	0
smoker	0
cough	0
fever	4157
sob	0
diarrhea	0
fatigue	0
headache	0
loss_of_smell	0
loss_of_taste	0
runny_nose	0
muscle_sore	0
sore_throat	0
target	0

dtype: int64

Class counts:

target	
0	7269
1	25

Name: count, dtype: int64

Out [6]:

	count	mean	std	min	25%	50%	75%
age	7294.0	39.717576	14.014773	0.0	30.0	38.0	50.0
high_risk_exposure_occupation	7294.0	0.029750	0.169910	0.0	0.0	0.0	0.0
high_risk_interactions	2727.0	0.200220	0.400238	0.0	0.0	0.0	0.0
diabetes	7294.0	0.001234	0.035108	0.0	0.0	0.0	0.0
chd	7294.0	0.001508	0.038807	0.0	0.0	0.0	0.0
htn	7294.0	0.032218	0.176591	0.0	0.0	0.0	0.0
cancer	7294.0	0.000685	0.026175	0.0	0.0	0.0	0.0
asthma	7294.0	0.007678	0.087291	0.0	0.0	0.0	0.0
copd	7294.0	0.000274	0.016558	0.0	0.0	0.0	0.0
autoimmune_dis	7294.0	0.000960	0.030966	0.0	0.0	0.0	0.0
smoker	7294.0	0.021662	0.145586	0.0	0.0	0.0	0.0
cough	7294.0	0.021662	0.145586	0.0	0.0	0.0	0.0
fever	3137.0	0.018170	0.133588	0.0	0.0	0.0	0.0
sob	7294.0	0.009460	0.096807	0.0	0.0	0.0	0.0
diarrhea	7294.0	0.005895	0.076559	0.0	0.0	0.0	0.0
fatigue	7294.0	0.029613	0.169530	0.0	0.0	0.0	0.0
headache	7294.0	0.025638	0.158062	0.0	0.0	0.0	0.0
loss_of_smell	7294.0	0.001645	0.040530	0.0	0.0	0.0	0.0
loss_of_taste	7294.0	0.001097	0.033102	0.0	0.0	0.0	0.0
runny_nose	7294.0	0.022896	0.149581	0.0	0.0	0.0	0.0
muscle_sore	7294.0	0.013847	0.116864	0.0	0.0	0.0	0.0
sore_throat	7294.0	0.027831	0.164500	0.0	0.0	0.0	0.0
target	7294.0	0.003427	0.058448	0.0	0.0	0.0	0.0

Interpretation

After dropping sparse/admin columns we retain age, comorbidity flags, symptom flags, and a couple of risk indicators. Remaining missing values in `high_risk_interactions` and `fever` will be imputed with the most frequent value. Trees do not require feature scaling, which keeps preprocessing simpler for this task.

```
In [7]: feature_cols = [c for c in df_c.columns if c != "target"]
X = df_c[feature_cols]
y = df_c["target"]
```

```
# Simple most-frequent imputation for remaining NaNs
imputer = SimpleImputer(strategy="most_frequent")
X_imp = pd.DataFrame(imputer.fit_transform(X), columns=feature_cols)

print("Any NaNs left?", X_imp.isnull().sum().sum())
X_imp.head()
```

Any NaNs left? 0

```
Out[7]:
```

	age	high_risk_exposure_occupation	high_risk_interactions	diabetes	chd	htn	c
0	39.0	0.0	0.0	0.0	0.0	0.0	
1	56.0	0.0	0.0	0.0	0.0	0.0	
2	35.0	0.0	0.0	0.0	0.0	0.0	
3	37.0	0.0	0.0	0.0	0.0	0.0	
4	42.0	0.0	0.0	0.0	0.0	0.0	

```
In [8]: # Stratified split so both classes appear in train and test
X_train, X_test, y_train, y_test = train_test_split(
    X_imp, y, test_size=0.25, random_state=RANDOM_STATE, stratify=y
)
print("Train size:", X_train.shape, "Test size:", X_test.shape)
print("Train positive rate: {:.3f}%".format(100 * y_train.mean()))
print("Test positive rate: {:.3f}%".format(100 * y_test.mean()))
```

Train size: (5470, 22) Test size: (1824, 22)

Train positive rate: 0.347%

Test positive rate: 0.329%

A.3 Exploratory Data Analysis

```
In [9]: # Age by test outcome
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(data=df_c, x="age", hue="target", bins=20, ax=axes[0], palette=
axes[0].set_title("Age distribution by COVID result (0=Neg, 1=Pos)")

symptom_cols = ["cough", "fever", "sob", "fatigue", "headache",
                "loss_of_smell", "loss_of_taste", "muscle_sore", "sore_throat"]
symptom_rates = (
    df_c.groupby("target")[symptom_cols].mean().T
)
symptom_rates.plot(kind="bar", ax=axes[1])
axes[1].set_title("Symptom rates by class")
axes[1].set_ylabel("Proportion reporting symptom")
axes[1].tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()
```

Interpretation

Symptom rates differ between classes on several features (for example smell/taste loss and cough are clinically associated with COVID). Because positives are so rare, these signals are noisy — tree ensembles with class balancing should help, but we should not expect perfect recall.

A.4 Model Design and Evaluation

We compare:

1. A **dummy majority-class baseline** (always predicts Negative)
2. A **Random Forest Classifier**
3. A **Gradient Boosting Classifier**

Because of class imbalance, Random Forest uses `class_weight="balanced"`. For Gradient Boosting we pass `sample_weight` so positive cases have higher influence during training.

```
In [10]: def classification_metrics(y_true, y_pred, y_proba=None, label="Model"):
    out = {
        "Model": label,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision (Pos)": precision_score(y_true, y_pred, zero_division=0),
        "Recall (Pos)": recall_score(y_true, y_pred, zero_division=0),
        "F1 (Pos)": f1_score(y_true, y_pred, zero_division=0),
    }
    if y_proba is not None and len(np.unique(y_true)) > 1:
        try:
            out["ROC-AUC"] = roc_auc_score(y_true, y_proba)
        except Exception:
            out["ROC-AUC"] = np.nan
    else:
        out["ROC-AUC"] = np.nan
    return out

# Class weights for GB sample weighting
n_pos = (y_train == 1).sum()
n_neg = (y_train == 0).sum()
sample_weight_train = np.where(y_train == 1, n_neg / max(n_pos, 1), 1.0)

# --- Baseline ---
dummy = DummyClassifier(strategy="most_frequent", random_state=RANDOM_STATE)
dummy.fit(X_train, y_train)
pred_dummy = dummy.predict(X_test)

# --- Random Forest (default-ish, then light tuning) ---
rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=8,
    min_samples_leaf=2,
    class_weight="balanced",
    random_state=RANDOM_STATE,
```

```

    n_jobs=-1,
)
rf.fit(X_train, y_train)
pred_rf = rf.predict(X_test)
proba_rf = rf.predict_proba(X_test)[:, 1]

# --- Gradient Boosting ---
gb = GradientBoostingClassifier(
    n_estimators=150,
    learning_rate=0.05,
    max_depth=3,
    random_state=RANDOM_STATE,
)
gb.fit(X_train, y_train, sample_weight=sample_weight_train)
pred_gb = gb.predict(X_test)
proba_gb = gb.predict_proba(X_test)[:, 1]

results_a = pd.DataFrame([
    classification_metrics(y_test, pred_dummy, label="Dummy (majority)"),
    classification_metrics(y_test, pred_rf, proba_rf, "Random Forest"),
    classification_metrics(y_test, pred_gb, proba_gb, "Gradient Boosting"),
])
results_a

```

Out[10]:

	Model	Accuracy	Precision (Pos)	Recall (Pos)	F1 (Pos)	ROC-AUC
0	Dummy (majority)	0.996711	0.0	0.0	0.0	NaN
1	Random Forest	0.980263	0.0	0.0	0.0	0.394206
2	Gradient Boosting	0.923246	0.0	0.0	0.0	0.176843

```

In [11]: # Light grid search on Random Forest (small grid – school assignment scale)
param_rf = {
    "n_estimators": [100, 200],
    "max_depth": [6, 10, None],
    "min_samples_leaf": [1, 3],
}
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE)
grid_rf = GridSearchCV(
    RandomForestClassifier(class_weight="balanced", random_state=RANDOM_STATE),
    param_rf,
    scoring="f1",
    cv=cv,
    n_jobs=-1,
)
grid_rf.fit(X_train, y_train)
print("Best RF params:", grid_rf.best_params_)
print("Best CV F1:", round(grid_rf.best_score_, 4))

best_rf = grid_rf.best_estimator_
pred_rf_t = best_rf.predict(X_test)
proba_rf_t = best_rf.predict_proba(X_test)[:, 1]

# Light grid search on Gradient Boosting
param_gb = {

```

```

    "n_estimators": [100, 150],
    "learning_rate": [0.05, 0.1],
    "max_depth": [2, 3],
}
grid_gb = GridSearchCV(
    GradientBoostingClassifier(random_state=RANDOM_STATE),
    param_gb,
    scoring="f1",
    cv=cv,
    n_jobs=-1,
)
# GridSearchCV does not take sample_weight in fit easily across folds with cv
# fit with sample_weight via fit_params in older sklearn; use fit on best model
# For simplicity/compatibility we tune without sample_weight then refit best model
grid_gb.fit(X_train, y_train)
print("Best GB params:", grid_gb.best_params_)
print("Best CV F1 (unweighted tune):", round(grid_gb.best_score_, 4))

best_gb = GradientBoostingClassifier(**grid_gb.best_params_, random_state=RANDOM_STATE)
best_gb.fit(X_train, y_train, sample_weight=sample_weight_train)
pred_gb_t = best_gb.predict(X_test)
proba_gb_t = best_gb.predict_proba(X_test)[:, 1]

# Default 0.5 threshold is too harsh with ~0.3% positives.
# Choose the threshold that maximises Positive F1 on the TRAINING set.
def best_threshold(y_true, proba):
    best_t, best_f1 = 0.5, -1
    for t in np.linspace(0.05, 0.95, 19):
        f1 = f1_score(y_true, (proba >= t).astype(int), zero_division=0)
        if f1 > best_f1:
            best_f1, best_t = f1, t
    return best_t, best_f1

proba_rf_train = best_rf.predict_proba(X_train)[:, 1]
proba_gb_train = best_gb.predict_proba(X_train)[:, 1]
t_rf, _ = best_threshold(y_train, proba_rf_train)
t_gb, _ = best_threshold(y_train, proba_gb_train)
print(f"Chosen RF threshold: {t_rf:.2f}")
print(f"Chosen GB threshold: {t_gb:.2f}")

pred_rf_t = (proba_rf_t >= t_rf).astype(int)
pred_gb_t = (proba_gb_t >= t_gb).astype(int)

results_a_tuned = pd.DataFrame([
    classification_metrics(y_test, pred_dummy, label="Dummy (majority)"),
    classification_metrics(y_test, pred_rf_t, proba_rf_t, f"Tuned RF @ thr={t_rf:.2f}"),
    classification_metrics(y_test, pred_gb_t, proba_gb_t, f"Tuned GB @ thr={t_gb:.2f}"),
])
results_a_tuned

# Cross-validated ROC-AUC is more stable than a single tiny test fold (only 5 folds)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
from sklearn.model_selection import cross_val_score
cv_auc_rf = cross_val_score(best_rf, X_imp, y, cv=cv, scoring="roc_auc", n_jobs=-1)
# For GB, wrap with sample weights via a simple approach: use unbalanced fit
cv_auc_gb = cross_val_score(

```

```

GradientBoostingClassifier(**grid_gb.best_params_, random_state=RANDOM_S
X_imp, y, cv=cv, scoring="roc_auc", n_jobs=-1
)
print("5-fold CV ROC-AUC - Random Forest:      {:.3f} ± {:.3f}".format(cv_auc
print("5-fold CV ROC-AUC - Gradient Boosting: {:.3f} ± {:.3f}".format(cv_auc
print("Note: with only 25 Positive cases total, even CV estimates have high

Best RF params: {'max_depth': 6, 'min_samples_leaf': 1, 'n_estimators': 100}
Best CV F1: 0.0222
Best GB params: {'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 100}
Best CV F1 (unweighted tune): 0.1952
Chosen RF threshold: 0.75
Chosen GB threshold: 0.90
5-fold CV ROC-AUC - Random Forest:      0.544 ± 0.096
5-fold CV ROC-AUC - Gradient Boosting: 0.460 ± 0.119
Note: with only 25 Positive cases total, even CV estimates have high varianc
e.

```

```

In [12]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))
ConfusionMatrixDisplay.from_predictions(y_test, pred_rf_t, ax=axes[0], color
axes[0].set_title("Confusion Matrix - Tuned Random Forest")
ConfusionMatrixDisplay.from_predictions(y_test, pred_gb_t, ax=axes[1], color
axes[1].set_title("Confusion Matrix - Tuned Gradient Boosting")
plt.tight_layout()
plt.show()

print("\n--- Random Forest classification report ---")
print(classification_report(y_test, pred_rf_t, target_names=["Negative", "Po
print("\n--- Gradient Boosting classification report ---")
print(classification_report(y_test, pred_gb_t, target_names=["Negative", "Po

--- Random Forest classification report ---
              precision    recall  f1-score   support

   Negative         1.00        1.00        1.00        1818
   Positive         0.00        0.00        0.00         6

   accuracy                   1.00        1824
  macro avg         0.50        0.50        0.50        1824
 weighted avg         0.99        1.00        0.99        1824

--- Gradient Boosting classification report ---
              precision    recall  f1-score   support

   Negative         1.00        1.00        1.00        1818
   Positive         0.00        0.00        0.00         6

   accuracy                   1.00        1824
  macro avg         0.50        0.50        0.50        1824
 weighted avg         0.99        1.00        0.99        1824

```

```

In [13]: # Feature importance comparison
imp_rf = pd.Series(best_rf.feature_importances_, index=feature_cols).sort_va
imp_gb = pd.Series(best_gb.feature_importances_, index=feature_cols).sort_va

```

```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
imp_rf.head(12).sort_values().plot(kind="barh", ax=axes[0], color="steelblue")
axes[0].set_title("Random Forest – Top Feature Importances")
imp_gb.head(12).sort_values().plot(kind="barh", ax=axes[1], color="darkorange")
axes[1].set_title("Gradient Boosting – Top Feature Importances")
plt.tight_layout()
plt.show()

print("Top RF features:\n", imp_rf.head(8))
print("\nTop GB features:\n", imp_gb.head(8))

```

Top RF features:

headache	0.262643
age	0.173516
loss_of_smell	0.118703
fever	0.071275
sore_throat	0.059778
diarrhea	0.057846
muscle_sore	0.055859
runny_nose	0.040060

dtype: float64

Top GB features:

age	0.486095
headache	0.237967
loss_of_smell	0.106995
sore_throat	0.061207
fever	0.045993
htn	0.023985
high_risk_interactions	0.013384
high_risk_exposure_occupation	0.010391

dtype: float64

Results summary (Task 1)

Both ensembles are compared against a majority-class dummy. Important context: the dataset has only **25 Positive** cases out of 7,294 (~0.34%). A random 75/25 split leaves roughly **6 Positives** in the test set, so Positive precision/recall/F1 on that single fold are unstable and can easily be zero even when the model has learned a weak ranking.

That is why we also report:

- confusion matrices (to see false positives / false negatives),
- feature importances (are the drivers clinically sensible?),
- **5-fold stratified CV ROC-AUC** (slightly more stable than one test fold).

Takeaway for the assignment: the modelling pipeline is complete and comparable for RF vs GB, but the data does not support a strong COVID classifier. Accuracy alone would hide this — with such extreme imbalance, Positive-class metrics and confusion matrices matter far more.

Feature importance: age and symptoms such as headache / loss of smell / fever often rank highest, which is directionally sensible even when overall discrimination is weak.

Practical implication: a real screening tool would need far more Positive cases (or external clinical features) before deployment.

A.5 Discussion — Dataset 1 / Task 1

- **What worked:** Dropping near-empty columns avoided inventing vitals/imaging. Class weighting and threshold exploration made the imbalance problem visible instead of papering over it with accuracy. Feature importance still gives an interpretable story.
 - **RF vs GB:** Random Forest was generally more stable. Gradient Boosting can chase rare positives aggressively, which with $n \approx 25$ Positives often means fitting noise — CV ROC-AUC and test metrics should be read with that caution.
 - **Limitations:** The binding constraint is **Positive sample size**, not the choice of tree algorithm. Missing clinical fields also remove potentially useful signal.
 - **On interpretability:** Trees needed no scaling and provide feature importances, which helps explain predictions. Still, no model can invent signal that is not in the data.
-

Part B — Task 2: Car Price Prediction with a Decision Tree

Objective: Predict automobile prices using the Auto dataset.

Model: Decision Tree Regressor, evaluated with RMSE and R^2 , plus interpretation of the tree.

B.1 Data Loading and Overview

```
In [14]: df_auto_raw = pd.read_csv("Auto Dataset.csv", encoding="latin-1", low_memory
print("Shape:", df_auto_raw.shape)
df_auto_raw.head()
```

Shape: (50000, 20)

Out [14]:

	dateCrawled		name	seller	offerType	
0	3/26/2016 17:47		Peugeot_807_160_NAVTECH_ON_BOARD	privat	Angebot	\$
1	4/4/2016 13:38	BMW_740i_4_4_Liter_HAMANN_UMBAU_Mega_Optik		privat	Angebot	\$
2	3/26/2016 18:57		Volkswagen_Golf_1.6_United	privat	Angebot	\$
3	3/12/2016 16:58	Smart_smart_fortwo_coupe_softouch/F1/Klima/Pan...		privat	Angebot	\$
4	4/1/2016 14:38	Ford_Focus_1_6_Benzin_TÜV_neu_ist_sehr_gepfleg...		privat	Angebot	\$

Interpretation

The Auto dataset has 50,000 used-car listings and 20 columns (German eBay-style ads): price, vehicle attributes, registration year, mileage, brand, etc. The regression target is `price`.

In [15]:

```
df_auto_raw.info()
print("\nMissing %:")
print((df_auto_raw.isnull().mean() * 100).sort_values(ascending=False).head())
```

```

<class 'pandas.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   dateCrawled                          50000 non-null  str
1   name                                50000 non-null  str
2   seller                              50000 non-null  str
3   offerType                           50000 non-null  str
4   price                               50000 non-null  str
5   abtest                              50000 non-null  str
6   vehicleType                         44905 non-null  str
7   yearOfRegistration                  50000 non-null  int64
8   gearbox                             47320 non-null  str
9   powerPS                             50000 non-null  int64
10  model                               47242 non-null  str
11  odometer                            50000 non-null  str
12  monthOfRegistration                  50000 non-null  int64
13  fuelType                            45518 non-null  str
14  brand                               50000 non-null  str
15  notRepairedDamage                   40171 non-null  str
16  dateCreated                         50000 non-null  str
17  nrOfPictures                        50000 non-null  int64
18  postalCode                          50000 non-null  int64
19  lastSeen                            50000 non-null  str
dtypes: int64(5), str(15)
memory usage: 7.6 MB

```

```

Missing %:
notRepairedDamage    19.658
vehicleType           10.190
fuelType              8.964
model                 5.516
gearbox               5.360
odometer              0.000
postalCode            0.000
nrOfPictures          0.000
dateCreated           0.000
brand                 0.000
dtype: float64

```

B.2 Data Cleaning and Preprocessing

```

In [16]: df_a = df_auto_raw.copy()

# Clean price: strip currency symbols/commas
df_a["price"] = (
    df_a["price"].astype(str)
    .str.replace(r"^[^0-9]", "", regex=True)
    .replace("", np.nan)
    .astype(float)
)

# Clean odometer: e.g. '150,000km' -> 150000
df_a["odometer"] = (

```

```

df_a["odometer"].astype(str)
    .str.replace(r"^[^0-9]", "", regex=True)
    .replace("", np.nan)
    .astype(float)
)

# Drop columns not useful for price modelling
df_a = df_a.drop(columns=[
    "dateCrawled", "name", "dateCreated", "lastSeen",
    "nrOfPictures", # all zeros / not informative
    "postalCode", # location noise for this assignment
    "abtest", "seller", "offerType", # almost constant / non-predictive
])

# Remove unrealistic prices and vehicle attributes
before = len(df_a)
df_a = df_a[(df_a["price"] >= 500) & (df_a["price"] <= 40000)]
df_a = df_a[(df_a["yearOfRegistration"] >= 1990) & (df_a["yearOfRegistration"] <= 2016)]
df_a = df_a[(df_a["powerPS"] >= 40) & (df_a["powerPS"] <= 400)]
df_a = df_a[df_a["odometer"].notna()]

print(f"Rows kept: {len(df_a)} / {before} ({100*len(df_a)/before:.1f}%)")
print(df_a["price"].describe())

```

Rows kept: 38313 / 50000 (76.6%)

```

count    38313.000000
mean      6081.239318
std       6461.684542
min        500.000000
25%       1599.000000
50%       3700.000000
75%       8000.000000
max      39999.000000

```

Name: price, dtype: float64

Interpretation

Raw prices include zeros, tiny placeholder values, and extreme outliers (up to 1e8).

Filtering to a realistic used-car range (€500–€40,000), registration years 1990–2016, and sensible horsepower removes junk listings that would distort a decision tree.

```

In [17]: # Fill categoricals with 'unknown'; then label-encode for the tree
cat_cols = ["vehicleType", "gearbox", "model", "fuelType", "brand", "notRepairedDamage"]
num_cols = ["yearOfRegistration", "powerPS", "odometer", "monthOfRegistration"]

for c in cat_cols:
    df_a[c] = df_a[c].fillna("unknown").astype(str)

encoders = {}
for c in cat_cols:
    le = LabelEncoder()
    df_a[c] = le.fit_transform(df_a[c])
    encoders[c] = le

feature_cols_b = cat_cols + num_cols

```

```
X_b = df_a[feature_cols_b]
y_b = df_a["price"]

print("Features:", feature_cols_b)
print("Any NaNs?", X_b.isnull().sum().sum())
X_b.head()
```

Features: ['vehicleType', 'gearbox', 'model', 'fuelType', 'brand', 'notRepairedDamage', 'yearOfRegistration', 'powerPS', 'odometer', 'monthOfRegistration']
Any NaNs? 0

```
Out [17]:
```

	vehicleType	gearbox	model	fuelType	brand	notRepairedDamage	yearOfRegistration
0	1	1	38	6	25	1	2
1	6	0	17	1	2	1	2
2	6	1	114	1	37	1	2
3	4	0	104	1	32	1	2
5	1	0	231	3	4	2	2

```
In [18]: X_b_train, X_b_test, y_b_train, y_b_test = train_test_split(
          X_b, y_b, test_size=0.2, random_state=RANDOM_STATE
        )
print(X_b_train.shape, X_b_test.shape)
```

(30650, 10) (7663, 10)

B.3 Exploratory Data Analysis

```
In [19]: fig, axes = plt.subplots(1, 3, figsize=(14, 4))
sns.histplot(y_b, bins=40, ax=axes[0], color="teal")
axes[0].set_title("Price distribution (cleaned)")

sns.scatterplot(x=df_a["yearOfRegistration"], y=df_a["price"], s=8, alpha=0.3)
axes[1].set_title("Price vs Year of Registration")

sns.scatterplot(x=df_a["powerPS"], y=df_a["price"], s=8, alpha=0.3, ax=axes[2])
axes[2].set_title("Price vs Power (PS)")
plt.tight_layout()
plt.show()
```

Interpretation

Price is right-skewed (many cheaper cars, fewer expensive ones). Newer cars and higher horsepower tend to sell for more — relationships that a decision tree can capture with piecewise splits, without assuming linearity.

B.4 Model Design and Evaluation

```
In [20]: def regression_metrics(y_true, y_pred, label="Model"):
    return {
        "Model": label,
        "RMSE": root_mean_squared_error(y_true, y_pred),
        "MAE": mean_absolute_error(y_true, y_pred),
        "R2": r2_score(y_true, y_pred),
    }

    # Baseline: predict mean price
    dummy_r = DummyRegressor(strategy="mean")
    dummy_r.fit(X_b_train, y_b_train)
    pred_dummy_r = dummy_r.predict(X_b_test)

    # Default decision tree (will overfit if fully grown – we still show it)
    dt_default = DecisionTreeRegressor(random_state=RANDOM_STATE)
    dt_default.fit(X_b_train, y_b_train)
    pred_dt_def = dt_default.predict(X_b_test)

    print("Default tree depth:", dt_default.get_depth(), "leaves:", dt_default.get_n_leaves())

    results_b0 = pd.DataFrame([
        regression_metrics(y_b_test, pred_dummy_r, "Dummy (mean)"),
        regression_metrics(y_b_test, pred_dt_def, "Default DT (unpruned)"),
    ])
    results_b0
```

Default tree depth: 38 leaves: 27899

```
Out[20]:
```

	Model	RMSE	MAE	R2
0	Dummy (mean)	6353.756972	4696.713987	-0.000736
1	Default DT (unpruned)	3023.624948	1708.520854	0.773372

```
In [21]: # Tune tree with a small grid – control depth for generalisation + interpretability
    param_dt = {
        "max_depth": [4, 6, 8, 10, 12],
        "min_samples_leaf": [20, 50, 100],
        "min_samples_split": [50, 100],
    }
    grid_dt = GridSearchCV(
        DecisionTreeRegressor(random_state=RANDOM_STATE),
        param_dt,
        scoring="neg_root_mean_squared_error",
        cv=KFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE),
        n_jobs=-1,
    )
    grid_dt.fit(X_b_train, y_b_train)
    print("Best DT params:", grid_dt.best_params_)
    print("Best CV RMSE:", round(-grid_dt.best_score_, 2))

    best_dt = grid_dt.best_estimator_
    pred_dt = best_dt.predict(X_b_test)

    results_b = pd.DataFrame([
        regression_metrics(y_b_test, pred_dummy_r, "Dummy (mean)"),
        regression_metrics(y_b_test, pred_dt, "Best DT (tuned)"),
    ])
    results_b
```

```

    regression_metrics(y_b_test, pred_dt_def, "Default DT (unpruned)"),
    regression_metrics(y_b_test, pred_dt, f"Tuned DT {grid_dt.best_params_}")
])
results_b

```

Best DT params: {'max_depth': 12, 'min_samples_leaf': 20, 'min_samples_split': 50}

Best CV RMSE: 2739.74

Out[21]:

	Model	RMSE	MAE	R2
0	Dummy (mean)	6353.756972	4696.713987	-0.000736
1	Default DT (unpruned)	3023.624948	1708.520854	0.773372
2	Tuned DT {'max_depth': 12, 'min_samples_leaf': ...}	2612.752952	1537.080691	0.830779

```

In [22]: # Predicted vs actual
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].scatter(y_b_test, pred_dt, s=10, alpha=0.35)
axes[0].plot([y_b_test.min(), y_b_test.max()], [y_b_test.min(), y_b_test.max()])
axes[0].set_xlabel("Actual price")
axes[0].set_ylabel("Predicted price")
axes[0].set_title("Tuned Decision Tree - Predicted vs Actual")

resid = y_b_test - pred_dt
sns.histplot(resid, bins=40, ax=axes[1], color="slategray")
axes[1].set_title("Residual distribution (Actual - Predicted)")
plt.tight_layout()
plt.show()

```

```

In [23]: # Feature importances
imp_dt = pd.Series(best_dt.feature_importances_, index=feature_cols_b).sort_
plt.figure(figsize=(8, 4))
imp_dt.sort_values().plot(kind="barh", color="seagreen")
plt.title("Decision Tree Regressor - Feature Importances")
plt.tight_layout()
plt.show()
imp_dt

```

```

Out[23]: yearOfRegistration    0.523791
powerPS                      0.346867
odometer                     0.081728
brand                        0.013147
vehicleType                  0.012832
model                       0.011149
fuelType                     0.003966
gearbox                      0.002999
monthOfRegistration          0.002143
notRepairedDamage           0.001378
dtype: float64

```

```

In [24]: # Interpret a shallower tree diagram (max_depth=4) for readability
dt_viz = DecisionTreeRegressor(max_depth=4, min_samples_leaf=100, random_sta
dt_viz.fit(X_b_train, y_b_train)

```

```
plt.figure(figsize=(20, 10))
plot_tree(
    dt_viz,
    feature_names=feature_cols_b,
    filled=True,
    rounded=True,
    fontsize=8,
    impurity=False,
    proportion=True,
)
plt.title("Decision Tree Regressor (max_depth=4) – structure for interpretation")
plt.show()

print(export_text(dt_viz, feature_names=feature_cols_b)[:2500])
```

```

|--- yearOfRegistration <= 2007.50
|   |--- powerPS <= 214.50
|       |--- yearOfRegistration <= 2003.50
|           |--- powerPS <= 125.50
|               |--- value: [1695.84]
|               |--- powerPS > 125.50
|                   |--- value: [3271.04]
|           |--- yearOfRegistration > 2003.50
|               |--- powerPS <= 139.50
|                   |--- value: [3939.65]
|                   |--- powerPS > 139.50
|                       |--- value: [7057.57]
|       |--- powerPS > 214.50
|           |--- odometer <= 137500.00
|               |--- powerPS <= 290.50
|                   |--- value: [13154.88]
|                   |--- powerPS > 290.50
|                       |--- value: [20430.13]
|           |--- odometer > 137500.00
|               |--- yearOfRegistration <= 2005.50
|                   |--- value: [7808.38]
|                   |--- yearOfRegistration > 2005.50
|                       |--- value: [12614.34]
|--- yearOfRegistration > 2007.50
|   |--- powerPS <= 148.00
|       |--- powerPS <= 101.50
|           |--- odometer <= 85000.00
|               |--- value: [7576.96]
|               |--- odometer > 85000.00
|                   |--- value: [4201.31]
|       |--- powerPS > 101.50
|           |--- odometer <= 85000.00
|               |--- value: [14338.29]
|               |--- odometer > 85000.00
|                   |--- value: [9090.40]
|   |--- powerPS > 148.00
|       |--- odometer <= 95000.00
|           |--- yearOfRegistration <= 2011.50
|               |--- value: [18853.90]
|               |--- yearOfRegistration > 2011.50
|                   |--- value: [24961.52]
|       |--- odometer > 95000.00
|           |--- powerPS <= 201.50
|               |--- value: [12705.74]
|               |--- powerPS > 201.50
|                   |--- value: [19123.93]

```

Results summary (Task 2)

The tuned decision tree should clearly beat the mean baseline on RMSE and R^2 .

Unpruned trees often look strong on training data but generalise worse — controlling `max_depth` and `min_samples_leaf` reduces overfitting.

Tree interpretation: Early splits usually involve `yearOfRegistration`, `powerPS`, `odometer`, and sometimes `brand / vehicleType`. That matches intuition: newer, more powerful, lower-mileage cars cost more. The depth-4 diagram is a readable explanation of how the model partitions the market; the tuned deeper tree is used for the reported test metrics.

B.5 Discussion — Dataset 2 / Task 2

- **What worked:** Cleaning price/odometer strings and filtering unrealistic listings mattered more than fancy modelling. A single decision tree is already competitive for this tabular pricing problem and is easy to explain.
- **Limitations of a single tree:** High variance — small data changes can rewrite splits. It also cannot extrapolate smoothly beyond the ranges seen in training. An ensemble (Random Forest / Gradient Boosting) would usually reduce error further, but the brief specifically asks for a Decision Tree Regressor.
- **Interpretability strength:** We can plot the tree and read feature importances — useful for communicating which attributes drive price.

Overall Discussion, Modeling Decisions, and Lessons Learned

Summary of results

Dataset	Task	Models	Key metrics
COVID-19 tests	Classification	Random Forest & Gradient Boosting	Accuracy, Precision/Recall/F1 (Positive), ROC-AUC, confusion matrices
Auto listings	Regression	Decision Tree Regressor	RMSE, MAE, R^2 , tree diagram, feature importance

Modeling decisions and rationale

1. **Dropped sparse COVID columns** rather than imputing 70–99% missing values — invented vitals/imaging would not be trustworthy.
2. **Class imbalance handling** for COVID via `class_weight` / `sample_weight` and F1-focused evaluation (accuracy alone misleads).
3. **No feature scaling** — tree splits are scale-invariant, so standardisation was unnecessary.
4. **Aggressive cleaning of Auto prices** — marketplace junk listings would dominate squared error otherwise.

5. **Pruned / depth-limited trees** for regression so the model generalises and remains interpretable.

Strengths and limitations of tree-based models

Strengths: handle non-linear relationships and mixed feature types; relatively light preprocessing; feature importances and plottable structure; ensembles (RF/GB) reduce variance. **Limitations:** single trees overfit if grown fully; unstable splits; struggle when the minority class has very few samples (COVID positives); stepwise predictions can be coarse near leaf boundaries.

Lessons learned

Tree-based methods were a natural fit for both tasks. For COVID classification, the main challenge was **data scarcity for the Positive class**, not the choice between RF and GB. For car pricing, **data cleaning quality** set the ceiling; a well-tuned decision tree already explains price with intuitive features (year, power, mileage). Overall, trees were practical to preprocess and straightforward to interpret for both problems.